



수학 시뮬레이터 제작 보고서

《시뮬레이터》

① 최종 결과물

② 원도우

③ 메모장

(예 : 이현진1.html, 이현진2.html으로 꼭 확장자 붙여서 저장)

수행평가

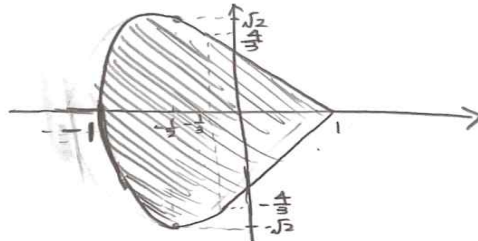
[첫 번째 시뮬레이터 제작 보고서]

1. 내가 선택한 첫 번째 문제 (캡처해서 붙여넣기 해도 됨)

부등식 $a\cos 2\theta + b\cos \theta < 1$ 이 모든 실수 θ 에 대하여 성립하고 있을 때, 점 (a,b) 의 범위를 도시하여라.

2. 이 문제의 풀이 (반드시 손글씨로 작성하여 캡처하거나 패드에 손글씨로 작성한 것을 붙여넣으세요.)

$$\begin{aligned}
 1. \quad & a(\cos 2\theta) + b\cos \theta < 1 \\
 \Rightarrow & a(2\cos^2 \theta - 1) + b\cos \theta < 1 \\
 & = 2a\cos^2 \theta - a + b\cos \theta - 1 < 0, \quad \cos \theta = t. \\
 & f(t) = 2at^2 + bt - a - 1 < 0, \quad -1 \leq t \leq 1 \\
 & \text{i) } a > 0 \\
 & \quad f(1) = a + b - 1 < 0 \quad f(-1) = a - b - 1 < 0 \\
 & \text{ii) } a < 0 \\
 & \quad f(t) = 2a\left(t + \frac{b}{4a}\right)^2 - 1 - \frac{b^2}{8a} - a < 0 \\
 & \text{ii-1) } -\frac{b}{4a} > 1 \\
 & \quad f(1) = a + b - 1 < 0 \\
 & \text{ii-2) } -\frac{b}{4a} < -1 \\
 & \quad f(-1) = a - b - 1 < 0 \\
 & \text{ii-3) } -1 \leq -\frac{b}{4a} \leq 1 \\
 & \quad -1 - \frac{b^2}{8a} - a^2 < 0 \Rightarrow \frac{(a + \frac{1}{2})^2}{4} + \frac{b^2}{2} < 1
 \end{aligned}$$



3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유를 서술하세요.

이 문제는 θ 에 대하여 부등식의 영역이 a, b 로 나타내어지기에 a, b 로 나타내지는 포락선의 영역을 파악하는 것이 중요하다. 각 θ 마다 ab 평면위에 하나의 경계선이 생성되고 이 선들의 집합이 곧 구하는 영역이 된다. 이처럼 무한한 조건의 누적과 포락선이라는 고급개념이 얹혀있어, 수식만으로는 영역의 형태를 직관적으로 파악하기 매우 어렵다. 시뮬레이션은 이 전 과정을 시각적으로 보조하는 필수 도구가 된다.

첫째, 무한한 θ 조건의 시각적 이해가 필요하다. (문제 파악)

모든 실수 θ 에 대해 성립이라는 조건은 무한히 많은 부등식을 동시에 만족시켜야 한다는 뜻이다. 특정 θ 값마다 ab 평면에 하나의 경계선이 그어지는데, 어떻게 회전하는지 눈으로 확인 할 수 있다.

둘째, 포락선이라는 개념 도출의 보조를 돕는다 (검토)

각 θ 에 대한 경계선들이 모여 이루는 포락선이 바로 점 (a,b) 범위의 실질적인 경계가 된다. 이 포락선의 형태를 수식만으로 예측하기는 매우 어렵지만, 시뮬레이터로 θ 를 연속적으로 변화시키며 경계선 묶음을 시각화하면 포락선의 정확한 곡선 형태가 자연스럽게 드러난다. 이를 통해 수식으로 계산한 포락선 방정식이 올바른지 즉각 검증할 수 있다.

셋째, 영역의 경계 형태와 대칭성을 파악하여 풀이의 결정적 단서를 얻는다. (풀이에 대한 힌트)

포락선으로 둘러싸인 허용 영역이 어떤 모양인지불록한지, 어떤 축에 대해 대칭인지, 꼭짓점이 존재하는지를 수식만으로 미리 그려내기는 어렵다. 시뮬레이터가 그려주는 영역을 보면 경계의 대칭성과 오목·볼록 구조가 즉각적으로 파악되어, 영역을 효과적으로 분할하여 도식하는 전략을 세울 수 있다.

4. 사용한 AI 플랫폼을 모두 적으세요.

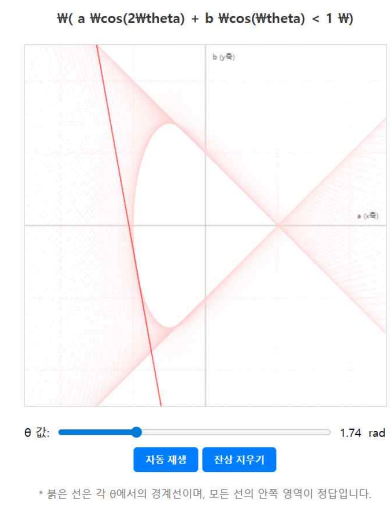
Gemini

5. 시뮬레이터를 만들기 위한 최초의 프롬프트를 작성하세요.

이 문제를 푸는 데 도움을 줄 수 있는 시뮬레이터를 html으로 만들자.

문제 상황을 정확히 그려주고, 세타를 내가 직접 움직일 수 있는 슬라이더와, 자동 재생되는 버튼을 함께 만들어 줘.

6. 최초 시뮬레이터의 모습을 캡처해서 붙여넣으세요.



7. 최초 시뮬레이터에서 수정해야 할 사항을 서술하세요.

최초 시뮬레이터는 선분이 조건에 맞게 움직이는 모습은 잘 구현했지만, 실제 문제 풀이에 직접적인 도움을 받기에는 다음과 같은 두 가지 부분에서 한계가 있었다.

첫째로 맨 위쪽 수식이 제대로 입력되지 않아서 깨졌다. 이는 다음 시뮬레이터에서 원본 수식으로 고치면 된다.

둘째, 각각 어떠한 도형을 만족하며 움직이는지 제대로 입력이 되지 않아, 전체적인 모양은 확인 할 수 있었지만, 포락선이 어떠한 도형의 방정식을 만족하는지에 대한 설명이 부족하여 알기 어려웠다.

셋째, 좌표축 사이에 숫자가 없어서 어떠한 단위로 움직이는지에 대한 정보가 부족했다.

8. 수정하기 위한 프롬프트를 (2번째 ~ n번째) 모두 작성하세요.

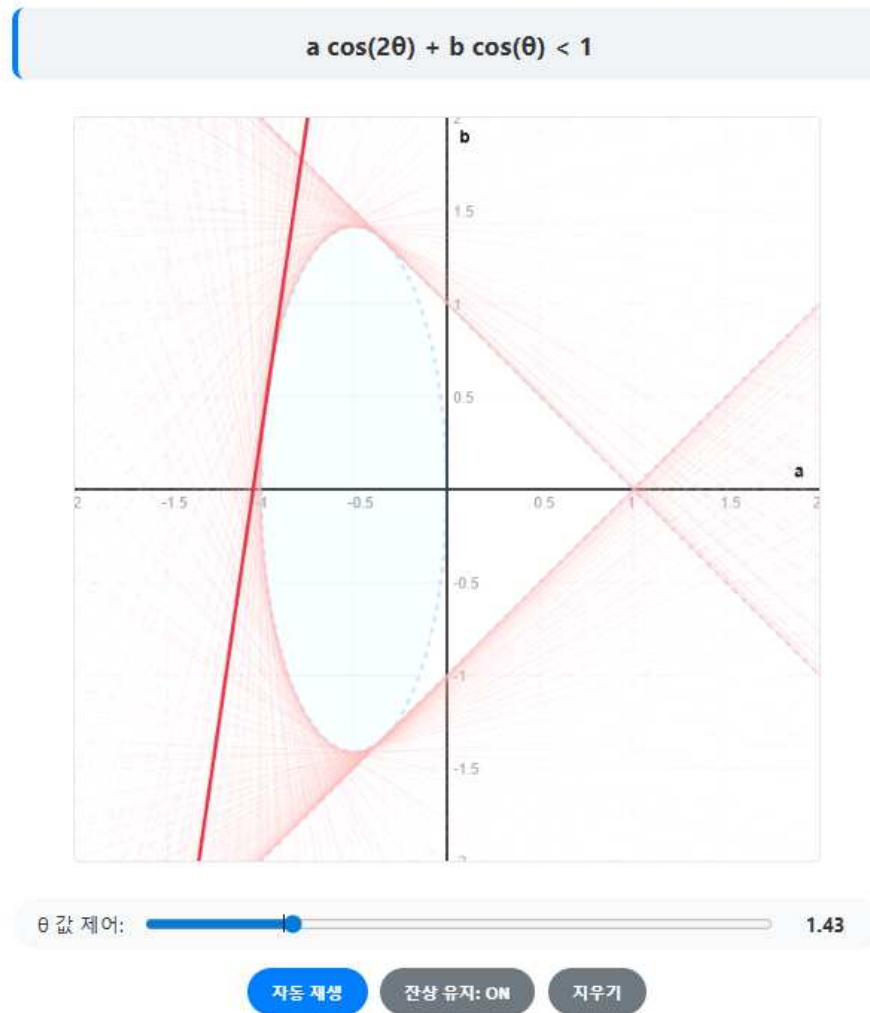
2번째 프롬프트

첫째로 위쪽의 수식이 깨졌어, 제대로 다시 입력해

둘째로 포락선이 어떠한 조건을 만족하며 움직이는지 알려줘

셋째 좌표축에 숫자 표시해줘

9. 최종 시뮬레이터의 모습을 캡처해서 붙여넣으세요.



[분석 결과]

- 경계 직선: $\theta = 0$ 일 때 $b = 1 - a$, $\theta = \pi$ 일 때 $b = a - 1$
- 포락선(곡선): $b^2 = -8a(a + 1)$ (타원의 일부)
- 해석: 위 모든 선들이 지나가는 한쪽의 공통 영역이 점 (a, b) 의 범위입니다.

10. 자신이 만든 시뮬레이터를 처음 본 사람이 사용할 수 있도록 사용 방법을 서술하세요.

주요 화면 설명

중앙 캔버스: (a,b) 좌표평면입니다. 가로축은 a , 세로축은 b 를 의미한다.

상단 수식: 현재 분석 중인 $a\cos 2\theta + b\cos \theta < 1$ 수식을 보여준다.

빨간색 직선: 현재 설정된 θ 값에서의 경계선 $a\cos 2\theta + b\cos \theta < 1$ 을 나타낸다.
이 직선을 기준으로 한쪽 영역만이 부등식을 만족한다.

파란색 점선 가이드: 이 문제의 정답이 될 포락선과 경계 직선들이다.

주요 컨트롤 기능

① θ 값 제어 (슬라이더)

방법: 슬라이더를 좌우로 움직인다.

의미: 각도가 변함에 따라 a 와 b 가 만족해야 하는 경계선이 어떻게 변하는지 확인할 수 있다.

특정 θ 에서 부등식을 만족하는 영역은 빨간 직선의 안쪽이다.

② 잔상 유지

방법: “잔상 유지” 버튼을 눌러 ON 상태로 만든다.

의미: 이 문제의 핵심은

모든 θ 에 대해 부등식이 성립해야 한다는 점이다.

잔상을 켜고 θ 를 변화시키면, 빨간 직선들이 지나간 자리가 화면에 계속 누적된다.

③ 자동 재생 & 지우기

자동 재생:

θ 가 0부터 2π 까지 자동으로 변하며 직선들의 움직임을 애니메이션으로 보여준다.

지우기:

화면에 쌓인 잔상을 모두 제거하고 초기 상태로 되돌린다.

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

[도움이 된 부분]

첫째, ‘모든 실수 θ ’라는 추상적 조건을 시각적으로 이해하는 데 큰 도움이 되었다.

단순히 수식만 보았을 때에는 (a,b) 의 범위가 어떻게 제한되는지 직관적으로 파악하기 어려웠다. 그러나 시뮬레이터의 ‘잔상 유지’ 기능을 사용하자, θ 가 변함에 따라 직선들이 좌표평면 위를 연속적으로 지나가는 모습이 누적되어 나타났고, 그 결과 직선들이 한 번도 침범하지 못하는 공통 영역이 존재한다는 사실을 쉽게 확인할 수 있었다. 이를 통해 정답 영역이 단순한 직선 사이의 구간이 아니라, 직선과 곡선이 함께 이루는 형태라는 점을 직관적으로 이해할 수 있었다.

둘째, 포락선의 존재라는 핵심 아이디어를 발견하는 데 도움이 되었다.

처음에는 경계가 단순히 $b = 1 - a$, $b = a - 1$

과 같은 직선들만으로 결정될 것이라고 생각하기 쉬웠다. 그러나 시뮬레이터에서 빨간 직선들이 회전하며 만들어내는 곡선 형태의 경계가 나타나는 것을 관찰하면서, 특정 구간에서는 직선이 아니라 곡선이 실제 경계 역할을 한다는 사실을 알 수 있었다. 이 관찰은 이후 “경계는 직선쪽의 포락선으로 주어진다”는 방향으로 사고를 확장하게 만들었고, 결국 $b^2 = -8a(a+1)$ 이라는 곡선 방정식을 유도해야 한다는 점을 깨닫게 해주었다.

셋째, 계산 결과를 기하학적으로 검토하는 데에도 유용했다.

예를 들어 계산 과정에서 얻은 결과들이 실제 시뮬레이터 화면의 점선 가이드 및 직선의 움직임과 일치하는지를 비교하면서 풀이의 타당성을 확인할 수 있었다. 따라서 시뮬레이터는 단순한 시각 자료를 넘어 계산 결과를 검증하는 보조 도구의 역할도 수행하였다.

[도움이 되지 않은 부분 (한계점)]

시뮬레이터는 문제의 ‘현상’을 보여줄 뿐, 이를 엄밀하게 증명하는 논리 구조까지 제공하지는 못했다. 화면에서는 곡선과 직선이 정답 영역의 경계처럼 나타나지만, 왜 그 경계가 형성되는지에 대한 수학적 이유는 직접 유도해야 했다. 예를 들어 $x = \cos\theta$ 로 치환하여 문제를 이차함수의 최대·최소 문제로 바꾸는 과정이나, 포락선을 구하기 위해 매개변수 θ 를 제거하는 과정 등은 사용자가 별도로 논리적으로 전개해야 했다. 또한 수치적·시각적 표현에는 엄밀성의 한계가 존재했다. 컴퓨터 그래픽은 픽셀 단위와 부동소수점 연산에 기반하기 때문에, 화면상으로는 직선이 곡선에 정확히 접하는 것처럼 보이더라도 실제로는 미세하게 교차하거나 떨어져 있을 가능성을 배제할 수 없다. 따라서 시각적 결과만으로 접한다고 결론 내릴 수는 없으며, 실제 서술형 풀이에서는 판별식과 같은 엄밀한 수학적 조건을 이용한 증명이 반드시 필요했다.

[한계점에 대한 대안]

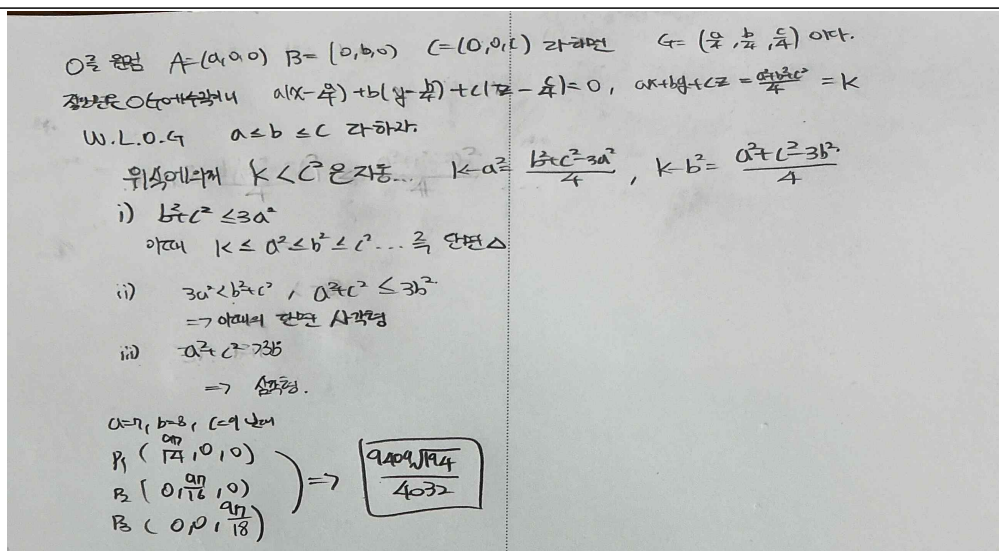
이러한 한계를 보완하기 위해서는 시뮬레이터를 ‘직관 형성 및 가설 검증 도구’로 활용하는 태도가 중요하다. 우선 시뮬레이터를 통해 정답 영역의 개형과 곡선 경계의 존재를 관찰한 뒤, “이 곡선은 직선의 포락선일 것이다”라는 가설을 세울 수 있다. 그 다음에는 $\cos 2\theta = 2\cos^2\theta - 1$ 과 같은 이배각 공식을 이용하여 식을 대수적으로 변형하고, 판별식 조건이나 포락선 계산을 통해 실제 경계식을 엄밀하게 유도해야 한다. 마지막으로 이렇게 얻어진 식이 시뮬레이터의 결과와 일치하는지를 다시 비교·검토함으로써, 직관과 증명을 연결하는 순환적인 학습 과정이 완성된다.

[두 번째 시뮬레이터 제작 보고서]

1. 내가 선택한 두 번째 문제 (캡처해서 붙여넣기 해도 됨)

사면체 $OABC$ 에 대하여, $\overrightarrow{OA}, \overrightarrow{OB}, \overrightarrow{OC}$ 는 서로 직교하고 있다. $\overrightarrow{OG} = \frac{1}{4}(\overrightarrow{OA} + \overrightarrow{OB} + \overrightarrow{OC})$ 인 점 G 를 지나고 $\overrightarrow{OG}$ 에 수직인 평면으로 자른 사면체 $OABC$ 를 자른 단면은 어떤 도형인가? (단 $|\overrightarrow{OA}| = a, |\overrightarrow{OB}| = b, |\overrightarrow{OC}| = c$ 라 하고 또한 $a = 7, b = 8, c = 9$ 일 때, 그 단면의 넓이를 구하여라.

2. 이 문제의 풀이 (반드시 손글씨로 작성하여 캡처하거나 패드에 손글씨로 작성한 것을 붙여넣으세요.)



3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유를 서술하세요.

이 문제에서 시뮬레이션이 필요하다고 느낀 이유는, 단면의 형태가 공간적으로 어떻게 만들어지는지를 머릿속으로 정확히 상상하기 어려웠기 때문이다.

문제에서는 사면체 $OABC$ 안을 특정 평면이 자르는데, 그 평면이 $\overrightarrow{OG} = \frac{1}{4}(\overrightarrow{OA} + \overrightarrow{OB} + \overrightarrow{OC})$ 를 지나는 동시에 $\overrightarrow{OG}$ 에 수직이라는 조건까지 포함하고 있습니다. 이러한 조건은 단순한 평면 절단이 아니라, 공간 속에서 기울어진 평면이 입체를 비스듬히 가로지르는 상황을 의미하므로, 실제 단면이 삼각형인지 사각형인지조차 직관적으로 판단하기 쉽지 않았다. 특히 이 문제의 핵심은 평면이 사면체의 어떤 모서리들과 만나는지를 파악하는 것이었는데, 이를 평면도만으로 이해하는 데 한계가 있었다. 평면이 네 개의 면 중 어디를 통과하는지, 그리고 절단 결과로 생기는 꼭짓점들이 어떻게 연결되는지를 머릿속으로만 추적하려다 보니 공간적 혼란이 생겼다. 따라서 3차원 시뮬레이션을 통해 평면의 위치와 단면의 모양을 직접 확인할 수 있다면, 단면이 삼각형인지 사각형인지와 같은 기본적인 개형을 훨씬 직관적으로 이해할 수 있다고 느꼈다. 또한 좌표를 설정하여 계산을 진행하더라도, 계산 결과가 실제 기하적 상황과 일치하는지 확신하기 어려웠다. 예를 들어 특정 교점을 구했을 때 그것이 실제로 사면체 내부에 존재하는 점인지, 혹은 단면이 실제로 닫힌 도형을 형성하는지를 식만으로는 즉시 판단하기 어려웠다. 반면 시뮬레이션에서는 평면과 사면체의 교차 모습을 바로 확인할 수 있기 때문에, 계산 과정에서 얻은 결과를 시각적으로 검토하는 데 큰 도움이 되었다. 결국 이 문제는 단순 계산보다 공간적인 구조를 이해하는 것이 핵심인 문제였기 때문에, 시뮬레이션이 풀이 방향을 설정하고 공간적 직관을 형성하는 데 효과적이라고 느꼈다.

4. 사용한 AI 플랫폼을 모두 적으세요.

Gemini

5. 시뮬레이터를 만들기 위한 최초의 프롬프트를 작성하세요.

이 문제를 푸는 데 도움을 줄 수 있는 시뮬레이터를 html으로 만들자.
문제 상황을 정확히 그려주고, a, b, c 값을 직접 슬라이더 형식으로 설정할 수 있게 바꾸고,
a = 7, b = 8, c = 9 인 케이스는 따로 버튼을 누르면 바로 나오게 해줘
그리고 문제를 풀기위한 가이드 라인도 만들어

6. 최초 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

사면체 단면 시뮬레이터

모서리가 직교하는 사면체 $OABC$ 를 점 G 를 지나고 OG 에 수직인 평면으로 자른 단면입니다.

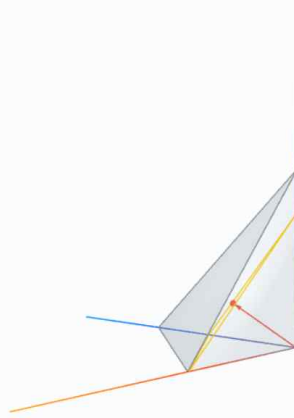
문제 조건 적용 ($a=7, b=8, c=9$)

매개변수 설정

a (OA 길이) 7
b (OB 길이) 8
c (OC 길이) 9

실시간 계산 결과

- 점 G : (1.75, 2.00, 2.25)
- 단면의 형태: **삼각형**
- 절편 X : 6.93 / Y : 6.06 / Z : 5.39
- 단면의 넓이: **32.5030**



마우스 드래그: 회전 | 휠: 확대/축소

7. 최초 시뮬레이터에서 수정해야 할 사항을 서술하세요.

- 문제를 어떻게 푸는지 잘 나타나지 않음
- 단면의 형태가 삼각형이 아니면 다각형으로 출력됨
- 어떤 점이 A, B, C인지 알기 어려움

8. 수정하기 위한 프롬프트를 (2번째 ~ n번째) 모두 작성하세요.

2번째

문제를 어떻게 푸는지 잘 나타나지 않음
단면의 형태가 삼각형이 아니면 다각형으로 출력됨
어떤 점이 A, B, C인지 알기 어려움, 이 3가지 고쳐

9. 최종 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

사면체 단면 시뮬레이터

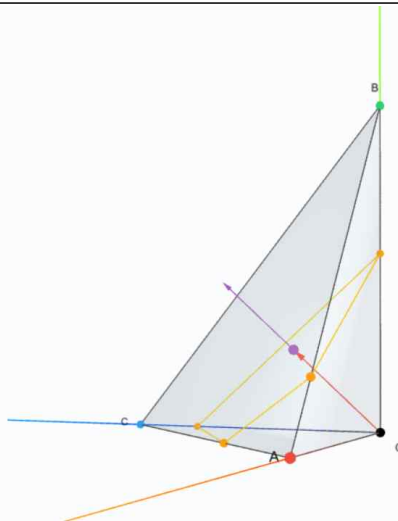
점 G 를 지나고 벡터 OG 에 수직인 평면으로 사면체 $OABC$ 를 절단합니다.

문제 조건 적용 (7,8,9)

a 7
b 10.5
c 9

실시간 계산 결과

- $G = (1.75, 2.63, 2.25)$
- 단면 형태: **사각형**
- 평면 방정식
 $x/8.58 + y/5.72 + z/6.67 = 1$
- 단면 넓이: **35.7322**



마우스 드래그: 회전
휠: 확대/축소

10. 자신이 만든 시뮬레이터를 처음 본 사람이 사용할 수 있도록 사용 방법을 서술하세요.

주요 화면 설명

① 중앙 3차원 화면

직교하는 사면체 $OABC$ 와 절단 평면을 시각적으로 보여주는 공간이다.

회색 반투명 입체는 사면체, 파란색 면은 단면을 의미한다.

마우스로 드래그하면 입체를 회전할 수 있으며, 휠로 확대·축소가 가능하다.

② 좌측 결과 패널

현재 설정된 a, b, c 값에 대한 계산 결과를 실시간으로 보여준다.

점 G 의 좌표, 단면의 형태, 절단 평면의 방정식, 단면의 넓이를 확인할 수 있다.

주요 컨트롤 기능

① a, b, c 값 제어 (슬라이더)

방법: 각 슬라이더를 좌우로 움직인다.

이를 통해 특정 조건에서 단면이 어떻게 달라지는지 탐구할 수 있다.

② 단면 분석

파란색 반투명 면은 실제 절단된 단면이다.

단면은 평면이 사면체의 어느 모서리를 통과하는지에 따라 여러 도형으로 변할 수 있다.

노란색 교점들을 통해 단면이 어떤 모서리들과 만나는지 직접 확인할 수 있다.

③ 벡터 및 평면 관계 확인

빨간 화살표 OG 와 보라색 법선벡터를 함께 표시하여, 조건을 직관적으로 이해할 수 있도록 하였다.

특히 보라색 법선벡터가 OG 와 같은 방향임을 통해, 단면 평면이 OG 에 수직이라는 사실을 시각적으로 확인할 수 있다.

④ 문제 조건 적용 버튼

문제에서 주어진 기본 조건 $a=7, b=8, c=9$ 를 즉시 적용하여 초기 상태로 되돌린다.

탐구 과정 중 값을 자유롭게 조절한 뒤 원래 조건으로 빠르게 복귀할 때 사용할 수 있다.

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

시뮬레이터는 단순히 계산 결과를 보여주는 것이 아니라, 공간도형 문제의 구조를 직관적으로 이해하는 데 큰 도움을 주었다. 특히 기존에는 평면의 방정식과 단면의 형태를 식으로만 해석해야 했기 때문에, 실제로 어떤 위치에서 평면이 사면체를 자르는지 상상하기 어려웠다. 그러나 시뮬레이터를 통해 점 G , 벡터 OG , 절단 평면, 그리고 단면의 교점들을 동시에 시각화할 수 있었고, 이를 통해 문제의 공간적 구조를 훨씬 쉽게 파악할 수 있었다.

가장 도움이 되었던 부분은 단면의 형태가 조건에 따라 달라진다는 점을 직접 확인할 수 있었다는 것이다. 처음에는 항상 삼각형 단면만 생성될 것이라고 생각했지만, a, b, c 값을 변화시키자 평면이 다른 모서리까지 통과하면서 사각형 이상의 다각형 단면이 나타날 수 있다는 사실을 발견할 수 있었다. 이를 통해 단순히 공식을 적용하는 것이 아니라, 평면이 어떤 모서리와 만나는지가 핵심이라는 점을 이해하게 되었다. 또한 노란색 교점 표시와 단면의 실시간 변화는 “왜 특정 조건에서 단면의 형태가 변하는가”를 스스로 탐구하는 데 도움이 되었다. 풀이 과정에 대한 힌트를 얻는 데에도 효과적이었다. 평면의 법선벡터가 OG 와 같은 방향이라는 사실을 빨간 화살표와 보라색 화살표를 통해 직관적으로 확인할 수 있었고, 이를 바탕으로 평면 방정식을 세우는 과정을 더 쉽게 이해할 수 있었다. 또한 단면의 넓이가 실시간으로 계산되기 때문에, 직접 유도한 식의 결과가 맞는지 비교하며 검증할 수 있었다. 실제로 삼각형 단면의 넓이를 계산한 뒤 시뮬레이터 값과 비교하면서 풀이가 올바른지 확인할 수 있었고, 계산 과정에서의 실수를 빠르게 발견할 수 있었다. 반면 한계점도 존재했다. 시뮬레이터는 시각적으로는 매우 직관적이지만, 왜 그런 공식이 성립하는지에 대한 수학적 증명 자체를 제공하지는 않는다. 따라서 이를 단순히 “정답 확인 도구”처럼 사용하면 원리를 깊이 이해하지 못할 가능성도 있다. 또한 3차원 화면을 회전시키지 않으면 특정 교점이나 단면 구조가 겹쳐 보이는 경우가 있어 처음 사용하는 사람에게는 다소 복잡하게 느껴질 수 있었다. 그럼에도 전체적으로는 공간도형 문제를 머릿속으로만 상상하는 부담을 줄여 주었고, 문제를 분석하고 풀이 방향을 떠올리며 결과를 검토하는 과정에서 매우 유용한 도구라고 느꼈다.

[마무리]

1. 두 문제에 대한 시뮬레이터를 기획하고 활용하여 문제를 해결하는 전 과정에서, 생성형 AI가 제공할 수 있는 가치와 반드시 인간(학생 본인)이 주도해야 하는 수학적 사고의 영역을 구분하여 서술하세요.

이번 탐구에서는 두 문제를 단순히 계산으로 해결하는 것이 아니라, 직접 시뮬레이터를 제작하여 문제의 구조를 시각적으로 분석하고 탐구하였다. 이 과정에서 생성형 AI는 코드 작성 보조, 오류 수정, 시각화 아이디어 제안 등에서 매우 큰 도움을 주었다. 특히 Three.js와 같은 라이브러리를 활용해 3차원 공간을 구현하거나, 복잡한 벡터 계산 및 단면 알고리즘을 코드로 변환하는 과정에서는 AI가 빠르게 예시 코드를 제공하여 개발 시간을 크게 단축시켜 주었다. 또한 시뮬레이터의 UI 구성이나 시각적 표현 방식에 대한 아이디어를 얻는 데에도 유용하였다.

그러나 문제를 실제로 이해하고 어떤 구조를 시뮬레이션해야 하는지를 결정하는 과정은 반드시 인간이 주도해야 한다는 점을 느꼈다. 예를 들어 사면체 단면 문제에서는 “평면이 어떤 모서리와 만나는가”, “왜 특정 조건에서 단면이 삼각형이 아닌 다각형이 되는가”와 같은 핵심 수학적 통찰은 직접 문제를 분석하며 얻어야 했다. AI는 계산이나 구현을 도와줄 수는 있었지만, 어떤 개념이 중요한지 판단하거나 탐구 방향 자체를 설정하지는 못했다. 또한 AI가 제시한 코드나 수식 역시 그대로 사용하는 것이 아니라, 실제 수학적 의미와 일치하는지 스스로 검증해야 했다. 따라서 생성형 AI는 복잡한 계산과 구현을 보조하는 강력한 도구가 될 수 있지만, 문제를 해석하고 핵심 아이디어를 발견하며 논리적으로 검증하는 과정은 인간의 수학적 사고가 반드시 중심이 되어야 한다는 점을 깨달았다.

2. 복잡한 수학 문제를 AI와 협업하여 시뮬레이터로 구현해 본 경험이, 향후 자신이 희망하는 진로 분야에서 실무적인 문제를 해결하는 데 어떤 시사점을 주는지 서술하세요.

이번 활동은 단순히 수학 문제를 푸는 경험을 넘어, 복잡한 문제를 분석하고 이를 디지털 환경에서 시각적으로 구현하는 경험이었다. 특히 수학적 개념을 코드로 변환하는 과정에서 “이론을 실제 시스템으로 구현하는 능력”의 중요성을 크게 느낄 수 있었다.

나는 인공지능 및 컴퓨터공학 분야에 관심이 있는데, 이번 경험을 통해 실제 실무에서도 단순한 프로그래밍 능력만으로는 충분하지 않다는 점을 알게 되었다. 예를 들어 AI 모델을 개발하거나 시뮬레이션 시스템을 설계할 때에도, 단순히 코드를 작성하는 것이 아니라 문제의 구조를 분석하고 핵심 원리를 이해해야만 올바른 결과를 만들 수 있다. 또한 AI가 제시한 결과를 그대로 받아들이는 것이 아니라, 논리적으로 검증하고 수정하는 능력이 매우 중요하다는 점도 느꼈다.

특히 이번 탐구에서 시뮬레이터를 제작하며 수학적 아이디어를 시각화한 경험은, 앞으로 복잡한 데이터를 직관적으로 표현하거나 실제 시스템을 모델링하는 분야에서도 큰 도움이 될 것이라고 생각한다. 또한 인간의 창의적 사고와 AI의 계산·구현 능력을 결합하면, 혼자서는 해결하기 어려운 문제도 더욱 효율적으로 탐구할 수 있다는 가능성을 확인할 수 있었다.

3. 보고서를 작성하며 느낀 점을 간단히 작성하세요.

이번 보고서를 작성하며 단순히 문제의 정답을 구하는 것보다, 문제의 구조를 직접 탐구하고 시각적으로 구현하는 과정이 훨씬 깊은 이해로 이어진다는 점을 느꼈다. 특히 공간도형 문제는 머릿속으로만 상상하기 어려웠는데, 시뮬레이터를 제작하면서 단면의 변화와 평면의 구조를 직관적으로 이해할 수 있었다.

또한 AI를 활용하면 구현 속도를 높이고 다양한 아이디어를 얻을 수 있었지만, 결국 중요한 것은 스스로 문제를 이해하고 판단하는 능력이라는 점도 깨달았다. 이번 활동은 수학, 프로그래밍, 시각화가 서로 연결될 수 있다는 점을 체험한 의미 있는 탐구였다.